



UNIVERSITÉ LIBRE DE BRUXELLES

Faculté des Sciences

BACHELIER EN SCIENCES INFORMATIQUES

INFOF307 : Génie logiciel et gestion de projets

Conventions de groupe

Projet 2025-2026

Professeur :
PLUQUET Frédéric

Client :
COLOT Martin

Coach :
ALNAKEB Derar

Groupe : 10

23 février 2026

Résumé

Ces conventions servent à rendre le code **cohérent, lisible et facile à maintenir** en équipe (pair programming, relecture, refactoring).

Les membres du groupe 10 pour le projet 2025-2026 en génie logiciel sont :

- AZEVEDO MOREIRA SILVEIRA CABEÇA Daniel (000543856)
- BANAITYTE Rugile (000588822)
- BLASIAK Daniel (000574901)
- EL HUSSEIN Abdalrahman (000596003)
- GULZAR Zia (000595624)
- HOUMACHE Wael (000584993)
- NADIF Sam (000594935)
- OMELYANYUK Oleksandra (000589880)
- SINON Clement (000382319)
- SMEETS Alexis (000592312)

Ce document se base sur les conventions de code Java d'Oracle¹

1. Conventions de code Java d'Oracle

Table des matières

1	Nommage	3
1.1	Fichiers et types	3
1.2	Identifiants	3
1.3	Langue (anglais vs français)	3
2	Organisation du code source	3
2.1	Structure MVC	3
2.2	Imports	4
3	Indentation et retours à la ligne	4
3.1	Indentation	4
3.1.1	Instructions simples	4
3.1.2	Instructions composées	4
3.1.3	Instruction <code>return</code>	4
3.1.4	Instructions <code>if</code> , <code>if-else</code> , <code>if-else-if-else</code>	5
3.1.5	Instructions <code>for</code>	5
3.1.6	Instructions <code>while</code>	5
3.1.7	Instructions <code>do-while</code>	5
3.1.8	Instructions <code>switch</code>	6
3.1.9	Instruction <code>try-catch</code>	6
3.2	Coupures de ligne (wrapping)	6
3.3	Espaces	6
4	Commentaires et Javadoc	7
4.1	Commentaires	7
4.2	Javadoc	7
5	Tests unitaires	8
5.1	Convention de nommage des tests unitaires	8
5.2	Décision de groupe	8
6	Git — format des commits	8

1 Nommage

1.1 Fichiers et types

- Un fichier `.java` contient **un seul type public**.
- Le nom du fichier correspond **exactement** au type public : `public class LoginController`
→ `LoginController.java`.
- PascalCase pour classes / interfaces / enums / records.

1.2 Identifiants

- **Classes** : noms (PascalCase), descriptifs (`GameBoard`, `ScoreCalculator`).
- **Interfaces** : nom de rôle (`UserRepository`), implémentations explicites (`InMemoryUserRepository`).
- **Méthodes** : verbes (camelCase) (`getScore`, `computePath`).
- **Variables** : camelCase (`currentPlayer`).
- **Constantes** : UPPER_SNAKE_CASE.

1.3 Langue (anglais vs français)

Pour garantir la cohérence, la lisibilité, et éviter les mélanges, la convention est d'écrire tout le code globalement en anglais, c-à-d :

- **Code (identifiants) en anglais** : noms de classes, méthodes, variables, constantes, packages.
- **Commentaires et Javadoc en anglais** lorsque c'est utile (contrat, préconditions, exceptions), afin de rester cohérent avec les API Java et les outils.
- **Exception** : si un concept est strictement propre au contexte du projet, p.ex. nom d'un Bugémon, noms des attaques, ..., alors ce sera en Français.

2 Organisation du code source

2.1 Structure MVC

À définir plus tard de façon plus concrète, mais apparemment :

- **model** : état et logique métier (entités, règles, services métier si vous les placez ici).
- **view** : UI (FXML, composants, vues).
- **controller** : orchestration (handlers d'événements, navigation, coordination entre view et model).
- **utils** : utilitaires génériques (éviter le "fourre-tout").
- La structure peut évoluer avec le projet, mais reste **cohérente** et documentée.

2.2 Imports

- Imports explicites : éviter `import x.*` sauf cas justifié.
- Supprimer les imports inutilisés.

3 Indentation et retours à la ligne

3.1 Indentation

- Tabs pour l'indentation.
- Une instruction par ligne. (cf prochaine sous-section)

3.1.1 Instructions simples

À éviter :

```
argv++; argc--; // AVOID!
```

À respecter :

```
argv++;  
argc--;
```

3.1.2 Instructions composées

Les instructions composées contiennent une liste d'instructions entourées d'accolades « { statements } ». Voir les sections suivantes pour des exemples.

- Les instructions contenues devraient être indentées d'un niveau supplémentaire par rapport à l'instruction composée.
- L'accolade ouvrante devrait être en fin de ligne qui commence l'instruction composée ; l'accolade fermante devrait commencer une ligne et être indentée au niveau du début de l'instruction composée.
- Des accolades sont utilisées autour de toutes les instructions, même lorsqu'il n'y en a qu'une, lorsqu'elles font partie d'une structure de contrôle (par ex. `if-else` ou `for`). Cela facilite l'ajout ultérieur d'instructions sans introduire de bogues en oubliant d'ajouter des accolades.

3.1.3 Instruction `return`

Une instruction `return` avec une valeur ne devrait pas utiliser de parenthèses, sauf si elles rendent la valeur retournée plus évidente. Exemple :

```
return;  
return myDisk.size();  
return (size ? size : defaultSize);
```

3.1.4 Instructions if, if-else, if-else-if-else

À respecter :

```
if (condition) {  
    statements;  
}  
if (condition) {  
    statements;  
} else {  
    statements;  
}  
if (condition) {  
    statements;  
} else if (condition) {  
    statements;  
} else if (condition) {  
    statements;  
}
```

À éviter :

```
if (condition) //AVOID! THIS OMITTS THE BRACES {}!  
    statement;
```

3.1.5 Instructions for

À respecter :

```
for (initialization; condition; update) {  
    statements;  
}
```

3.1.6 Instructions while

À respecter :

```
while (condition) {  
    statements;  
}
```

3.1.7 Instructions do-while

À respecter :

```
do {  
    statements;  
} while (condition);
```

3.1.8 Instructions switch

À respecter :

```
switch (condition) {
    case ABC:
        statements;
        /* falls through */
    case DEF:
        statements;
        break;
    case XYZ:
        statements;
        break;
    default:
        statements;
        break;
}
```

Chaque fois qu'un cas fait un « fall-through » (c.-à-d. qu'il ne contient pas d'instruction **break**), ajoutez un commentaire à l'endroit où le **break** devrait normalement se trouver. Ceci est illustré dans l'exemple précédent avec le commentaire `/* falls through */`.

Chaque instruction **switch** devrait inclure un cas **default**. Le **break** dans le cas **default** est redondant, mais il évite une erreur de fall-through si un nouveau cas est ajouté plus tard.

3.1.9 Instruction try-catch

À respecter :

```
try {
    statements;
} catch (ExceptionClass e) {
    statements;
}
```

3.2 Coupures de ligne (wrapping)

- Couper après une virgule, ou avant un opérateur.
- Aligner la ligne suivante sur un niveau logique identique, ou indenter d'un niveau supplémentaire si nécessaire.
- Viser **100 caractères max** par ligne. Au-delà, couper proprement.

```
function(longExpr1, longExpr2, longExpr3,
        longExpr4, longExpr5);

if ((cond1 && cond2)
    }) (cond3 && cond4)
    }) !cond5) {
    doSomething();
}
```

3.3 Espaces

Les espaces devraient être utilisés dans les cas suivants :

- Un mot-clé suivi d'une parenthèse devrait être séparé par un espace. Exemple :

```
while (true) {
    ...
}
```

Remarque : on ne met pas d'espace entre le nom d'une méthode et sa parenthèse ouvrante. Cela aide à distinguer les mots-clés des appels de méthode.

- Un espace devrait apparaître après les virgules dans les listes d'arguments.
- Tous les opérateurs binaires, sauf `.`, devraient être séparés de leurs opérandes par des espaces. Les espaces ne devraient jamais séparer des opérateurs unaires (moins unaire, incrémentation `++`, décrémentation `--`) de leurs opérandes. Exemple :

```
a += c + d;
a = (a + b) / (c * d);
while (d++ = s++) {
    n++;
}
prints("size is " + foo + "\n");
```

- Les expressions dans une instruction `for` devraient être séparées par des espaces. Exemple :

```
for (expr1; expr2; expr3)
```

- Les conversions de type (casts) devraient être suivies d'un espace. Exemples :

```
myMethod((byte) aNum, (Object) x);
myFunc((int) (cp + 5), ((int) (i + 3)) + 1);
```

4 Commentaires et Javadoc

4.1 Commentaires

- Commenter **l'intention** (le pourquoi), pas répéter le code.
- Éviter les commentaires qui vieillissent vite.
- Pas de “cadres” décoratifs en ASCII.

4.2 Javadoc

- Décrire le contrat : préconditions, effets, exceptions, contraintes.
- Compléter systématiquement `@param`, `@return`, `@throws` quand présents.

```
/**
 * Calcule le score total du joueur.
 *
 * @param player joueur concerné
 * @return score total
 * @throws IllegalArgumentException si player est null
 */
public int computeScore(Player player) { ... }
```


5 Tests unitaires

5.1 Convention de nommage des tests unitaires

Nom des classes de test

- Une classe de test teste **une** classe de production (en général).
- Nom standard : `NomDeLaClasseTest`.
- Le **package de test** reflète le package de production (même structure), sous `src/test/java`.

Exemples :

```
src/main/java/model/ScoreCalculator.java
src/test/java/model/ScoreCalculatorTest.java
```

Nom des méthodes de test (d'après les TPs)

Utiliser des verbes suivi des conditions à tester, p.ex. :

```
@Test
public void shouldEqual25() {
    assert ... == 25 : "Le résultat devrait être 25";
}

@Test
public void verifyCondition() {
    assert ... : "Message d'erreur";
}
```

5.2 Décision de groupe

- Pas de tests pour les **getters/setters** et les **constructeurs triviaux**.
- Tester un constructeur s'il est **complexe** (logique, validation) ou s'il **lance des exceptions**.
- Tout bug corrigé doit idéalement être accompagné d'un **test de régression**.

6 Git — format des commits

- Message: `<keyword>: <message court>` (ex: `add: écran de login, edit: validation mot de passe, remove: code mort`).
- Pair programming : mentionner le co-auteur via la ligne `Git VCo-authored-by: Prénom Nom <email>`.